Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Mitigating Class Imbalance in Pump-and-Dump Detection: A Comparative Analysis of Imbalance-Aware Algorithms

M. MIRONOV[1], D. BOGUTSKY[2], and P. LUKIANCHENKO[3]

[1]National Research University Higher School of Economics, Faculty of Computer Science, Laboratory of AI in Mathematical Finance (e-mail: mvmironov@hse.ru)
[2]National Research University Higher School of Economics, Faculty of Computer Science, Laboratory of AI in Mathematical Finance (e-mail: dbogucki@hse.ru)
[3]National Research University Higher School of Economics, Faculty of Computer Science, Laboratory of AI in Mathematical Finance (e-mail: plukyanchenko@hse.ru)

Corresponding author: D. Bogutsky (e-mail: dbogucki@hse.ru).

This work was supported by the grant for research centers in the field of AI provided by the Ministry of Economic Development of the Russian Federation in accordance with the agreement 000000C313925P4E0002 and the agreement with HSE University № 139-15-2025-009

ABSTRACT The cryptocurrency market has been subject to various forms of exploitation, with pump-and-dump schemes being a persistent problem that can lead to significant price volatility and financial losses. Previous studies have focused on predicting attack targets using machine learning approaches; however, these methods are often compromised by class imbalance and biases inherent in historical data. To address this limitation, we present a novel framework for predicting manipulation targets. We employ two-step bias-minimizing data normalization techniques to mitigate the effects of class imbalance and temporal heterogeneity. We also propose several innovative machine learning approaches including imbalance-aware hyperparameter optimization and ranking algorithms. Comparison-validation Top@k%AUC selects logistic regression (0.599) from seven tuned models. Frozen before this run's test evaluation, it attains test Top@1 of 0.058, Top@10 of 0.596, and Top@k%AUC of 0.769 (95% event-bootstrap interval $[0.681, 0.846]$) on 52 test events with approximately 1:297 class imbalance. Paired tests with Holm correction establish some model differences but no unique statistical winner. The 517-event study uses separate training, tuning, comparison, and test partitions; earlier inspection of these periods makes it a retrospective evaluation. We use the comparison-selected logistic model for the portfolio backtest and assess it under slippage-aware execution.

INDEX TERMS Classification, Imbalanced Datasets, Learning to rank, Machine Learning, Market Manipulation, Market Microstructure, Portfolio strategy

I. INTRODUCTION

A. WHAT ARE PUMPS AND DUMPS

PUMP-AND-DUMP schemes (P&Ds) are a type of fraudulent activity that involves artificially inflating the price of a cryptocurrency through coordinated buying efforts, followed by a rapid selling to unsuspecting investors. This phenomenon is often characterized by an abrupt and significant increase in trading volume, followed by a subsequent decline in price as the original buyers liquidate their positions.

These schemes typically involve experienced traders and market manipulators who orchestrate the operation through various channels, including online forums and social media platforms such as Telegram channels. The manipulation is designed to create a false sense of urgency and scarcity, which attracts inexperienced investors who may be unaware of the true nature of the event.

A typical pump-and-dump scenario can be observed in Fig. 1, which shows a clear example of this phenomenon. The figure illustrates price movements associated with a specific token VIB traded against BTC on Binance exchange, highlighting the abrupt spike in trading activity following the announcement of the token's release. This sudden increase in volume is often followed by a rapid decline in price as insiders sell their positions to new buyers.

B. RELATED WORK

The phenomenon of pump-and-dump schemes has been extensively studied in the context of cryptocurrencies, with

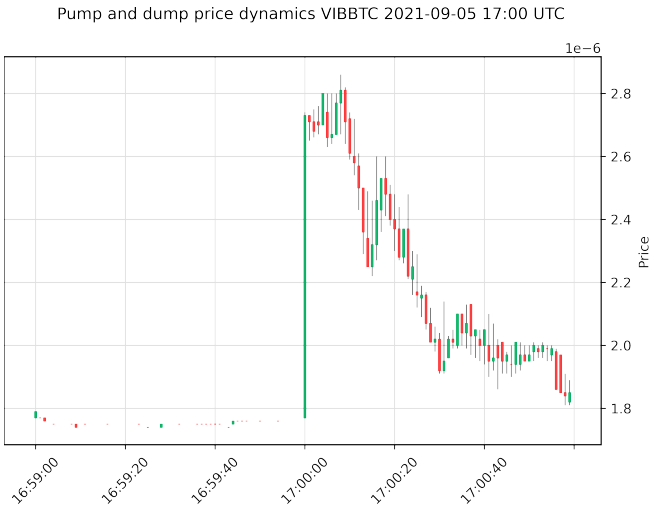


FIGURE 1. VIBBTC pump and dump manipulation carried out on Binance

research efforts spanning multiple approaches and methodologies.

One of the pioneering studies on this topic was [9], which introduced a comprehensive framework for understanding pump-and-dump operations. The authors highlighted the targeting of small market cap cryptocurrencies due to their lower trading volumes, utilizing data from 5 prominent crypto exchanges: Binance, Bittrex, Kraken, Kucoin, and Lbank. Their analysis covered pumps history of 1.5 years, with price and volume data being used as the primary sources. Subsequent research efforts have focused on leveraging machine learning (ML) algorithms to enhance P&D detection accuracy. La Morgia et al.'s work [10], [11] employed random forest algorithms, achieving improved results compared to traditional rule-based approaches. The authors in [4] further increased the effectiveness of La Morgia's work by incorporating Deep Neural Networks (LSTMs and Transformers) as anomaly detection algorithms. More recently, in [6] the authors built on these findings by studying a similar pump dataset between 2021 and 2022. They developed an innovative approach that employed resampling methods, such as SMOTE, combined with price and volume features and random forest classification to detect pump events 1 hour in advance.

In addition to pump detection, other research has focused on predicting the maximum price move during known pump events. Nghiem et al. [12] used market data and social sentiment with deep neural networks (LSTMs and CNNs) and achieved promising results. However, their analysis highlighted a potential limitation of using sentiment signals: historical bias may be introduced due to the lack of precise snapshots of historical social data.

A relevant example of work addressing the problem of predicting price manipulation targets is the study [16]. This research proposed an approach using random forest algorithms to identify potential pump and dump schemes among listed tokens. The authors analyzed a dataset of 180 pump events from the Cryptopia exchange, incorporating features such as price and volume data, global market capitalization data from Coinmarketcap, and token ratings by exchange. To enhance model robustness, the study employed filtering techniques to exclude pumps with suspicious announcement times that deviated significantly from full hours. Moreover, the authors validated their classification model through backtesting a portfolio strategy based on predicted pump targets. However, it is worth noting that the employed trading strategy relied on unrealistic assumptions, such as guaranteed profits of half the maximum price move during the pump and zero losses for false positive predictions.

A related study was conducted in [8], where authors enhanced their approach by incorporating channel embeddings into their model. They also utilized standard price and volume metrics, as well as external data sources such as market capitalization and social media information (e.g., Twitter followers, Reddit subscribers, Alexa rank). The analysis was performed on a dataset comprising 445 pump announcement events. However, due to the presence of duplicate pump events across multiple channels, the actual number of unique pump events studied is smaller than expected. Furthermore, the use of social data from different sources raises concerns about historical bias, as these metrics may have changed after the fact, potentially affecting the accuracy of the results.

Regarding market manipulation in a broad sense, several papers have also been published recently, for example [5], which studied the manipulation of trading volumes, also known as wash trading.

With the increasing popularity of ML applications in finance, in recent years we have seen many articles on predictions using the market microstructure, for example [2] and [13]. Promising results have also been published with applications of ranking models to finance [14]. Taking into account the rich accumulated experience in related areas, we aim to complement existing work with new factors based on the market microstructure.

In the broader ML literature, several families of techniques have been proposed for handling class imbalance beyond resampling, and each maps imperfectly onto the P&D target-prediction problem. Cost-sensitive learning [24] re-weights the loss so that misclassifying the minority class is more expensive; it does so without perturbing the feature space, which, as we show empirically in Section IV, avoids synthesizing observations in the cross-sectionally normalized feature space. Focal loss [26], originally proposed for dense object detection, down-weights easy examples and concentrates the gradient on hard, informative ones; it has since been adopted for imbalanced tabular classification, but remains most effective when combined with deep models and large minority classes, conditions that do not apply to our 310 positive training events. Synthetic oversampling methods such as SMOTE [17] have known pathologies in high-dimensional settings [21], which provide possible explanations for the results in Section IV.

Anomaly detection provides a complementary formulation

that does not require balanced training data. Isolation Forest [27] and related one-class approaches have been applied to fraud and market-abuse detection when positive examples are essentially unavailable. In our setting, however, positive examples *are* available (at least in limited numbers) and the task is explicitly *cross-sectional*: we do not ask "is this observation anomalous in isolation?" but rather "which asset in this cross-section is the most likely manipulation target?". This ranking-within-cross-section structure makes the problem closer to learning-to-rank than to unsupervised anomaly detection, and motivates the imbalance-aware ranking metric (Top@k%AUC) that we use for model comparison and most hyperparameter searches. To the best of our knowledge, no prior work on P&D target prediction has combined cross-sectional panel normalization, microstructure features, and an imbalance-aware ranking objective explicitly tailored to the 1-positive-vs.-many-negatives structure of each cross-section.

This paper builds upon previous work in [16] and [8], which focused on predicting manipulation targets among traded assets. We identify several critical limitations in data preparation that limit model reliability and propose innovative solutions to address these issues. Specifically, we introduce novel microstructure-based features leveraging insights from areas of market prediction [2], [13] and wash trading detection [5]. Moreover, we explore the use of boosted trees, as implemented in CatBoost [15], which has been shown to outperform other algorithms on tabular data (e.g., [7]), and we compare their performance to the random forest model introduced in [3].

To further improve the classification models' performance, we also perform a cross-sectional normalization method for panel data. Moreover, we formulate a specialized metric tailored to optimize the classification model for highly unbalanced datasets, which was used for model training and validation.

Table I summarizes the key methodological differences between our framework and the most closely related prior studies.

II. DATA COLLECTION AND PREPROCESSING

A. LABELED P&DS DATA

The released event manifest, `resources/pumps.json`, contains 517 unique announced P&D targets on Binance spot BTC pairs between January 3, 2018 and August 14, 2022. Each record supplies the target pair, exchange, and UTC announcement timestamp. These announcement labels are the fixed input to the empirical pipeline; no event is retained or discarded according to its subsequent return.

a: Channel identification.

We identified eight candidate Telegram channels through a two-step procedure. First, we searched Telegram directly using the keywords "crypto pump," "pump signal," "pump and dump," and their localized equivalents, retaining public channels whose stated purpose was the coordination of pump events. Second, we cross-referenced the resulting list with channels named or cited in prior literature [6], [10], [16]. For each channel, we exported the full message history (JSON format) using the Telegram Desktop client's built-in export functionality. The channel names cannot always be redistributed, but the timestamps and tickers of the retained events are in the public event file.

b: Labeling procedure.

The operational label record contains one row per {ticker, exchange, announcement timestamp}; there are no exact or ticker–timestamp duplicates. The manifest fixes the labels used computationally. Supplemental files `pump_event_evidence.json` and `pump_event_exclusions.json` document reviewed additions and exclusions, but the original sample has not undergone a complete independent annotation audit. Reproducing the computation therefore does not independently validate every announcement label.

c: Inclusion criteria.

A manifest event enters a model sample only if *all* of the following data-availability conditions hold:

- the target is a Binance spot pair quoted in BTC;
- the target has raw trade data in the exact past-only eligibility interval $[T - 15\text{min} - 1\text{d}, T - 15\text{min})$;
- a feature cross-section can be materialized from the local Binance archive.

d: Exclusion criteria.

No post-announcement price or volume threshold is used for sample inclusion. If a manifest event cannot be materialized because the required raw target history is absent, `create_dataset()` records the event in `resources/dropped_pumps.json`. This file is a reproducible missing-feature artifact; it does not encode social-media rejection reasons.

We performed the following preprocessing steps:

1) parse and validate every manifest record;
2) discover candidate BTC pairs only from date partitions intersecting the past eligibility interval;
3) inspect actual trade timestamps rather than date-partition membership, and require at least one trade in $[T - 15\text{min} - 1\text{d}, T - 15\text{min})$ for each candidate and the announced target;
4) materialize features and split complete event cross-sections chronologically.

The resulting dataset contains 517 unique market manipulation events, with no missing-feature exclusions in the current run. Restricting the analysis to Binance BTC pairs is a deliberate design choice with known trade-offs: (i) we lose events that targeted USDT-quoted pairs, tokens listed only on smaller venues (e.g., KuCoin, Gate, LBank, Cryptopia before its 2019 shutdown), and decentralized-exchange pumps; (ii) in exchange, we obtain full trade-level history for every pair in the cross-section, which is the minimum requirement

TABLE I. Comparison of the proposed framework with closely related prior work on P&D target prediction

Study	Exchange	# Events	Model	Imbalance Handling	Backtesting
Xu & Livshits [16]	Cryptopia	180	Random forest	None	Unrealistic assumptions
Hu et al. [8]	Multiple	445	Seq. model + channel embed.	None	None
Fantazzini & Xiao [6]	Binance	Varies	Random forest	SMOTE	None
La Morgia et al. [10]	Multiple	1111	Random forest	None	None
Ours	Binance	517	CatBoost, LR, RF, Ranker	Cross-sectional norm., Top@k%AUC opt., class weighting	Impact-aware execution

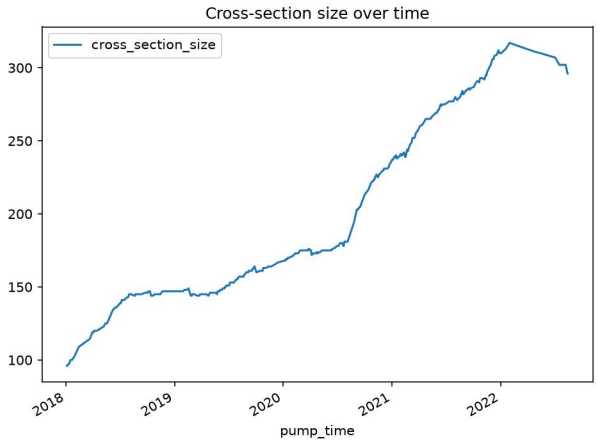


FIGURE 2. Binance has been increasing the number of tickers traded against BTC by listing more tokens.

for computing slippage, flow-imbalance and power-law features consistently across the whole sample; (iii) the restriction is aligned with the most comparable prior work on Binance P&D target prediction [6] and on microstructure-based detection [10], so our results remain directly comparable. We further discuss the limits of external validity in Section VI. Our data presents a substantial challenge, with P&D events occurring relatively infrequently compared to normal trading activity. During our study period, the average eligible cross-section contains 194.15 instruments quoted in BTC. The historical dynamics of the number of listings is shown in Fig. 2.

We split whole events chronologically into 60% training, 10% tuning, 20% comparison validation, and 10% testing, keeping simultaneous events together. Training ends at July 10, 2020 16:00:07 UTC; tuning runs from July 10, 2020 17:00:00 to January 2, 2021 21:00:00; comparison runs from January 3 to August 22, 2021; and testing runs from August 24, 2021 to August 14, 2022. Tuning and early stopping never use the comparison or test partitions. Table II describes the original event samples before training-only SMOTE.

	Train	Tuning	Comparison	Test
Positive	310	52	103	52
Negative	47158	11155	26107	15441
Total	47468	11207	26210	15493
Avg. cross-section size	153.12	215.52	254.47	297.94

TABLE II. Chronological event partitions (60/10/20/10).

B. MARKET DATA

To enable comparison with existing research, we selected Binance as the sole exchange platform. We use Binance's public historical trade archive for every BTC-quoted pair needed by the 2018–2022 event manifest and its 30-day feature lookbacks.

A representative example of our collected data is presented in Table III.

C. FEATURE ENGINEERING

We collected trade-level data as compressed daily archives for the required Binance pairs and dates. Because trading activity varies substantially across tickers and over time, the raw event streams are irregular and cannot be used directly as fixed-size inputs to standard machine-learning models. We therefore implemented a feature-generation pipeline that (i) decompresses and loads the relevant archives, (ii) partitions data into time-aligned windows, (iii) computes microstructure features summarizing pre-announcement order flow, and (iv) aggregates ticker-level features into an event-level cross-section associated with each pump. The pipeline is:

1) Download trade-level data from Binance's public daily archive.
2) For each pump event, construct the cross-section from tickers with at least one trade in the exact interval $[T - 15\text{min} - 1\text{d}, T - 15\text{min})$. Partition membership alone is insufficient because a daily partition can contain trades later than the decision time.
3) For each eligible ticker, load 30 days of history ending at the decision cutoff and compute features over prede-

	price	qty	time	isBuyerMaker	quote
0	0.000002	2437.00	2022-07-15 15:00:02.422000	True	0.004240
1	0.000002	275.00	2022-07-15 15:00:02.434000	False	0.000479
2	0.000002	1125.00	2022-07-15 15:00:22.884000	True	0.001946
3	0.000002	941.00	2022-07-15 15:00:52.277000	True	0.001628
4	0.000002	184.00	2022-07-15 15:00:52.277000	True	0.000318

TABLE III. Structure of collected data: *price* shows the price that the order was filled at, *qty* is the amount of base asset traded, *isBuyerMaker* indicates if the buyer's order related to this transaction was already in the orderbook: if True, the trade is classified as a Sell event and if False the trade is characterized as a Buy event.

fined half-open lookback windows $[T-15\text{min}-x, T-15\text{min})$. Hourly normalizer bars are re-anchored to the cutoff so every included bar is strictly pre-decision.

4) Combine features for all tickers into a single event-level dataset and assign labels by adding *is_pumped*, which equals 1 for the pumped ticker and 0 otherwise.

This modular pipeline allowed fast iteration over alternative feature sets. The implementation and reproduction instructions are available in [18]. We group features into several categories, described below.

1) Imbalance features

Volume imbalance measures which side is more prevalent in trade data. It is the ratio of signed notional in the quote asset (negative for seller-initiated trades and positive for buyer-initiated trades) to total absolute quote notional. Values near 1 indicate predominantly buyer-initiated flow and values near -1 predominantly seller-initiated flow.

$$\text{Volume Imbalance} = \frac{\sum_{t\in\mathcal{T}} s_t p_t q_t}{\sum_{t\in\mathcal{T}} p_t q_t}, \tag{1}$$

where $p_t q_t$ is quote notional and $s_t \in \{-1, 1\}$ is the taker-side sign at time t.

2) Quote slippages

We also created features based on slippage. Since we do not have information about order types, we cannot determine whether a trade was executed via a market or limit order; therefore, we treat slippage as a useful proxy for measuring market participants' impatience. We follow the methodology used in prior work by grouping ticks by execution time. Given that we deal with highly illiquid markets—where the time between trades can be as long as 15 minutes—we can reasonably assume that aggregated ticks correspond to a single order. This allows us to estimate the price impact of the aggregated order. The intuition is as follows: if a market participant is willing to incur large slippage losses by submitting a large buy order, they likely want to build their position quickly, potentially in anticipation of the pump. Based on this idea, we compute slippage for each aggregated trade using the formula in (2) below:

$$\begin{aligned} L &= \left| \sum_{i=1}^{N} q_i P_i - P_0 \sum_{i=1}^{N} q_i \right|, \\ L_{\text{signed}} &= L \ \text{sign}\left(\sum_{i=1}^{N} s_i q_i \right). \end{aligned} \tag{2}$$

where q_i, P_i, and s_i are respectively the unsigned base quantity, execution price, and taker-side sign of the ith print in an aggregated timestamp group, and P_0 is its first execution price. Thus L is the absolute difference between observed quote notional and the notional implied by filling the whole group at its first price; the sign is attached from net buyer- or seller-initiated base quantity. The slippage-imbalance feature is $\sum L_{\text{signed}} / \sum L$. As an order consumes liquidity it can appear as a sequence of prints at worsening prices, so this statistic is intended to proxy how aggressively the group walks available liquidity.

3) Arithmetic return features

For each horizon x, the implementation computes the arithmetic return $10^4(p_{\text{last}}/p_{\text{prev}} - 1)$ over $[T-15\text{min}-x, T-15\text{min})$. Return z-scores compare that window with the mean and sample standard deviation of decision-anchored hourly returns over the preceding 30 days. For $x \in \{5, 15\}$ minutes, the exact-window return is converted to an hourly rate before standardization; for horizons of at least one hour, complete cutoff-anchored hourly bars are averaged. Thus the 5-minute, 15-minute, and 1-hour columns are distinct, while every numerator and normalizer remains strictly pre-decision.

4) Power law features. Quantile spread of order sizes

To summarize the tail of aggregated trade notionals $x \geq x_{\min}$ within each lookback window, we use the Pareto density

$$f(x; \alpha, x_{\min}) = (\alpha - 1) x_{\min}^{\alpha-1} x^{-\alpha}, \qquad \alpha > 1, \tag{3}$$

where $x_{\min}$ is the minimum positive aggregated notional observed in the window. The code uses the closed-form maximum-likelihood estimate

$$\hat{\alpha} = 1 + \frac{n}{\sum_{i=1}^{n} \log(x_i / x_{\min})}. \tag{4}$$

Smaller $\hat{\alpha}$ indicates a heavier notional tail. Before cross-sectional standardization, the implementation clips $\hat{\alpha}$ to the

closed interval $[1, 2]$. This deterministic bound handles degenerate short windows in which the denominator is zero (and the raw estimate is infinite) and limits the influence of extremely sparse tails. This is a fixed model-preprocessing rule, not a sample-dependent 95% winsorization; no histogram-density regression or log–log OLS is performed.

All of the features discussed are further explained below (see Table IV):

D. CROSS-SECTIONAL APPROACH

Given that our event dataset spans early 2018 through August 2022, we account for temporal heterogeneity by applying cross-sectional standardization. This approach reduces market-regime level shifts and focuses the models on relative cross-asset dynamics.

We apply cross-sectional standardization to the unbounded return, return-z-score, quote-volume-z-score, and power-law features. For each selected feature, we subtract its mean within an event and divide by its within-event sample standard deviation:

$$X_{\text{standardized}} = \frac{X - \bar{X}_{CS}}{\bar{\sigma}_{CS}} \tag{5}$$

If a selected feature is constant within an event, it contains no relative ranking information and is mapped to zero; all-missing event columns remain missing for native handling or train-only imputation. We do not apply this second-stage transform to bounded imbalance/share features or to count features (trade counts and lifetime prior-pump count), whose nonnegative levels retain their direct interpretation. Fig. 3 demonstrates the transform on a sample of five cross-sections for three selected numerical features:

a: Checks for lookahead and regressor leakage. We verified the cutoff $t_0 = T - 15\text{min}$ both by inspecting input windows and by perturbing future data. Automated tests run the actual trade discovery, loading, aggregation, and feature writer twice: first with past-only trades, then with extreme prices and volumes at and after t_0, future-only listings, and later pump announcements. Candidate membership and all 81 declared regressors must remain exactly identical, while the deliberately forward-looking return labels change. Cases cover midnight, non-hour-aligned announcements, and subsecond boundaries. A deliberately calendar-aligned normalizer changes under the same perturbation, confirming that the test detects the historical lookahead mechanism. Separate provenance checks cover the 30-day normalizers and lagged prices; prior-pump counts exclude announcements at or after t_0.

Additional tests change held-out features, labels, and future returns and check that training imputation statistics, earlier-event standardization, and model-input matrices remain unchanged. Only declared predictors enter the estimator; `is_pumped`, return targets, asset identifiers, and return ranks are excluded from X. Instrumented fitting and tuning tests reject access to comparison/test rows, and SMOTE is restricted to training. Execution tests verify first-trade-at-or-after fills, completed conversion bars, and impact windows ending strictly before the relevant execution. These checks found no leakage along the tested paths; they do not constitute a universal proof about annotation or all possible implementation errors.

III. MODELLING

In this section, we attempt to create a model able to label this fraudulent activity 15 minutes before the announcement. As discussed earlier this is a challenging task due to high imbalance in the data. The task of finding manipulated assets can be designed as a binary classification problem where "1" implies that the asset is pumped and "0" that it is not. Therefore, we trained logistic regression, random forest, and CatBoost classifiers. The problem can also be formulated as learning to rank: the CatBoost ranker is supervised by each asset's realized arithmetic return over the first five minutes after announcement, ranked highest-first within its event cross-section (ordinal position 1 is the highest realized return). These post-announcement returns are labels only; the ranker's regressors are the same strictly pre-decision microstructure features used by the classifiers.

A. EVALUATION METRICS

Top@k is the probability that the ground-truth target occurs among a model's k highest scores. In portfolio terms, it is the probability that a size-k portfolio contains the pump target. Top@k% replaces the fixed count with a percentage of each cross-section, controlling for the growth in Binance listings described in Section II-A. We use one notation convention throughout: Top@k and Top@k% denote the fixed-count and percentage hit-rate curves; Top@kAUC and Top@k%AUC denote their respective areas. A concrete threshold replaces k in a point label, so Top@10 is the top-10 hit rate and Top@5% is the top-5% hit rate. The reported and optimized area in this study is Top@k%AUC; Top@kAUC is reserved for an area over fixed-count k and is not reported here.

The area under the Top@k% curve (Top@k%AUC) serves as our primary optimization target. In our setup we integrate the Top@k% curve over the low-$k\%$ region $k\% \in (0, 20\%]$ and normalize by the length of the integration range so the metric remains in $[0, 1]$:

$$\text{Top@k\%AUC} = \frac{1}{0.20}\int_0^{0.20} \text{Top@k\%}\, d(k\%). \tag{6}$$

Restricting integration to the $(0, 20\%]$ region is motivated by two considerations. First, it is the economically relevant region for a concentrated P&D portfolio: a practical strategy holds at most the top 10–20% of the cross-section in any given event. Second, all reasonably-trained models saturate above $k\% \approx 30\%$ and their Top@k% curves converge, so integrating over the full $(0, 1)$ range dilutes the metric with a large k region where every model is close to 1 and no longer discriminates between them. Truncating at 20%

Feature	Description	Notation
Asset return	return of the asset calculated over the window of size (T-15min-x, T-15min) prior to the pump announcement	*asset_return*[x]@t
Standardized return	exact-window hourly-rate return (5m, 15m) or mean cutoff-anchored hourly return ($x \geq$ 1h), minus the 30-day hourly mean and divided by its standard deviation	*asset_return_zscore*[x]@t
Standardized volume in BTC	exact-window hourly-rate volume (5m, 15m) or mean cutoff-anchored hourly volume ($x \geq$ 1h), standardized by 30-day hourly moments	*quote_abs_zscore*[x]@h
Share of long trades	share of long trades as the ratio of number of long trades and all trades of this time window (T-15min-x, T-15min)	*share_of_long_trades*[x]@h
Number of trades	number of timestamp-aggregated trades within $[T - 15\text{min} - x, T - 15\text{min})$	*num_trades*[x]@h
Flow imbalance ratio	volume imbalance ratio computed over the window of size (T-15min-x, T-15min)	*flow_imbalance*[x]@h
Quote slippage imbalance ratio	imbalance ratio of slippages in BTC calculated over the same time window	*slippage_imbalance*[x]@t
Powerlaw alpha	estimated alpha from powerlaw probability density function using the window of size (T-15min-x, T-15min) prior to the pump	*powerlaw_alpha*[x]@h
Number of previous pumps	lifetime number of manifest events for this token strictly before the current announcement	*num_prev_pump*

TABLE IV. These are the features included in all models. These features are used to describe historical dynamics of price and trading volume in the run up to the pump. $x \in \{$5*m*, 15*m*, 1*h*, 2*h*, 4*h*, 12*h*, 1*d*, 2*d*, 7*d*, 14*d*$\}$

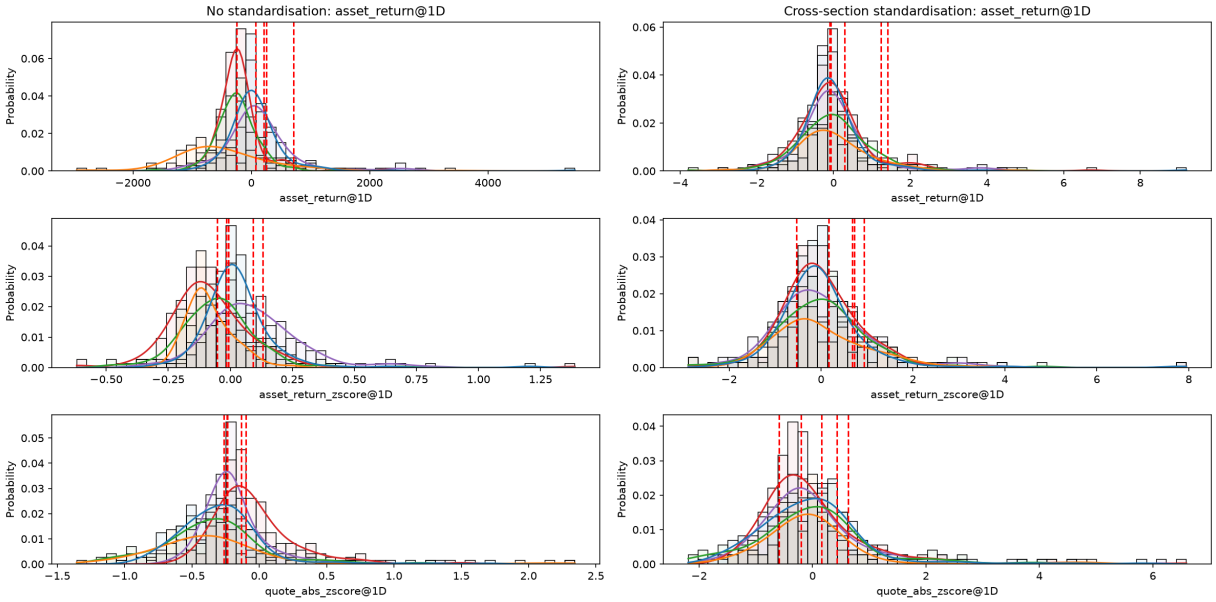


FIGURE 3. Features with and without cross-sectional standardization: histograms and kernel-density estimates for five P&D events. Applying standardization helps to align distributions more closely with one another across pumps.

makes Top@k%AUC materially more sensitive to differences between models and to hyperparameter choices, which is the property we want from both an Optuna tuning target and an early-stopping signal inside CatBoost. Numerically, Top@k%AUC values reported in this paper are therefore not directly comparable to AUCs integrated over $(0, 1)$; a model that saturates quickly below 20% can attain a Top@k%AUC close to 0.8 under this convention.

We use $K = \lceil fN_e \rceil$ assets at fraction f in event e, zero hits at $f = 0$, and trapezoidal integration on a 0.005 grid. Events receive equal weight; tied scores follow a fixed input order.

1) Limitations of Top@k-based metrics

While Top@k metrics are directly relevant for portfolio construction (how many assets must we hold to catch the pump?), they have limitations. They do not measure overall classi-

fication quality across the full score distribution and can mask poor calibration. A model may achieve high Top@k by ranking the pump target above most non-targets without producing well-separated probabilities. To address this, we complement Top@k with standard classification metrics.

2) Standard classification metrics

We additionally report three standard metrics on both the validation and test sets:

- **PR-AUC:** Area under the precision–recall curve, which is more informative than receiver operating characteristic (ROC) AUC under extreme class imbalance [24].
- **F1-score:** Harmonic mean of precision and recall, computed using a ''Top@1 per cross-section'' decision rule (predicting the highest-scored asset as positive).
- **Balanced accuracy:** Average of sensitivity and specificity, providing a class-imbalance-aware accuracy measure.

B. BASELINE MODEL

We use logistic regression as a simple baseline and compare it with a random-ranking model that selects candidate targets uniformly within each event cross-section. Logistic regularization and minority-class weighting are tuned on the tuning partition. Our primary evaluation metrics are Top@k with $k \in \{1, 2, 3, 5, 10, 20, 30\}$ and Top@k%AUC, analyzed in Section IV.

C. TRAINING

We fit all models on the training partition. Tree depth is constrained for random forest and CatBoost. CatBoost classifiers and the return ranker use the separate tuning partition for early stopping; CatBoost + ES monitors the custom Top@k%AUC metric with 50 rounds of patience and restores its best iteration.

Each of the seven pipelines underwent 100 Optuna trials using only the tuning partition. Most searches maximize tuning Top@k%AUC; the additional CatBoost + Top-5 model minimizes an equal-event hinge loss against the fifth-highest negative score, using the same tuning partition for stopping. After fitting the candidates, we select the largest Top@k%AUC point estimate on the independent comparison partition and freeze that model for test metrics and portfolios. Test outcomes do not tune or select models in this run. These calendar periods were inspected in earlier research, so the repartition is retrospective rather than a newly unseen prospective holdout.

On the training partition only, we also used SMOTE proposed by [17] to tackle the problem of class imbalance. It allows to create new observations for the minority class to balance the dataset. Instead of copying existing minority points, it generates new, slightly different ones by interpolating between neighbors.

a: Reproducibility.

The run identifier is `split_60_10_20_10_v1`. Its dataset manifest records event IDs, UTC boundaries, and SHA-256 hashes for all 517 feature files; saved model artifacts and Optuna studies retain parameters and split metadata. Fixed regressor order, candidate ordering, and seeds (42 for tuning, estimators, and event bootstraps) make repeated evaluation deterministic. The research notebook loads the saved best tuning parameters; a fresh search uses `backtest.research.retune` with 100 trials and a new study database. Reproduction requires the event manifest, the same Binance archive/features, source revision, dependency environment, and studies or fitted models; seeds alone are insufficient.

The `test_pump_bootstrap` exports include fixed predictions, per-event contributions, every sampled pump ID (including repeated draws), replicate metrics, confidence intervals, paired comparisons, and settings. We verified the feature/model hashes and independently replayed all 10,000 test resamples from those exports: all four metrics, percentile intervals, and 84 Holm decisions agreed (numerical tolerance 10^{-14}). The manuscript's `results_20261016/provenance.json` records the verification environment and test command; 158 targeted leakage, isolation, execution, determinism, and bootstrap tests passed. Same-artifact replay is distinguished from retraining, for which bitwise equality across hardware or library versions is not guaranteed.

IV. RESULTS

A. VALIDATION PERFORMANCE AND MODEL SELECTION

Table V reports the seven-model comparison used for final selection. Logistic regression has the highest comparison Top@k%AUC (0.599), narrowly ahead of CatBoost (0.597), and is frozen for the test portfolio. The difference is 0.0021 (paired 95% interval $[-0.0501, 0.0539]$; Holm-adjusted $p = 1.000$). Selection by the point estimate therefore does not establish superiority over CatBoost. CatBoost leads comparison Top@1 (0.136), whereas logistic regression leads the broader-ranking AUC. Section IV-I reports the full paired inference.

Fig. 4 shows validation Top@k against the validation-specific random-ranking baseline, while Fig. 5 shows validation Top@k% curves and their Top@k%AUC values. Fig. 6 provides the corresponding validation classification comparison. Together with Table V, these figures make the selection decision auditable without consulting the test set.

B. TOP@K ACCURACY

After freezing logistic regression as the portfolio model on comparison validation, we evaluate all seven candidates on the test partition. Test comparisons do not alter that selection.

Table VI shows that model ordering depends on portfolio size. CatBoost + ES leads Top@1 (0.212), CatBoost leads Top@2 (0.365) and Top@5 (0.462), and logistic regression leads Top@10 (0.596), Top@20 (0.827), and Top@30

TABLE V. Comparison-validation metrics for the seven tuned models. Top@k%AUC selects the portfolio model; bold marks column maxima, not statistical significance. CatBoost + ES denotes Top@k%AUC early stopping. F1 and balanced accuracy use Top@1 per event.

	Top@k%AUC	Top@1	Top@5	Top@10	Top@1%	Top@5%	Top@10%	PR-AUC	F1	Balanced accuracy
Logistic regression	**0.599**	0.078	0.311	0.447	0.214	0.485	**0.670**	0.040	0.078	0.537
CatBoost classifier	0.597	**0.136**	**0.320**	**0.466**	0.223	**0.524**	0.660	0.052	**0.136**	**0.566**
CatBoost + ES	0.551	0.097	0.291	0.427	**0.233**	0.466	0.592	0.042	0.097	0.547
CatBoost + Top-5	0.514	0.058	0.301	0.417	**0.233**	0.466	0.534	**0.062**	0.058	0.527
Random forest	0.487	0.058	0.194	0.301	0.126	0.379	0.515	0.034	0.058	0.527
CatBoost ranker	0.472	0.029	0.282	0.379	0.175	0.388	0.515	0.029	0.029	0.513
CatBoost + SMOTE	0.461	0.058	0.194	0.272	0.155	0.379	0.485	0.036	0.058	0.527

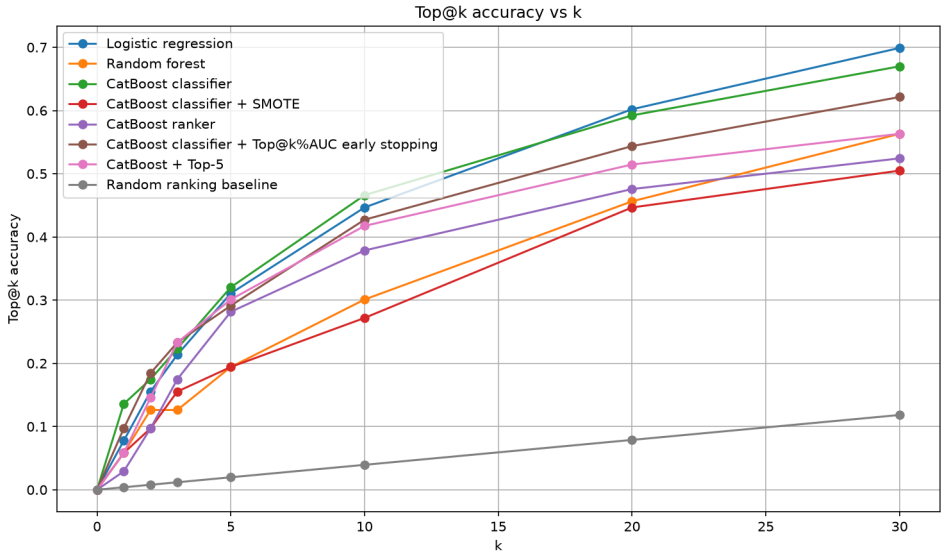


FIGURE 4. Top@k accuracy on the comparison partition for all seven tuned models and its random-ranking baseline. This partition is excluded from fitting and early stopping.

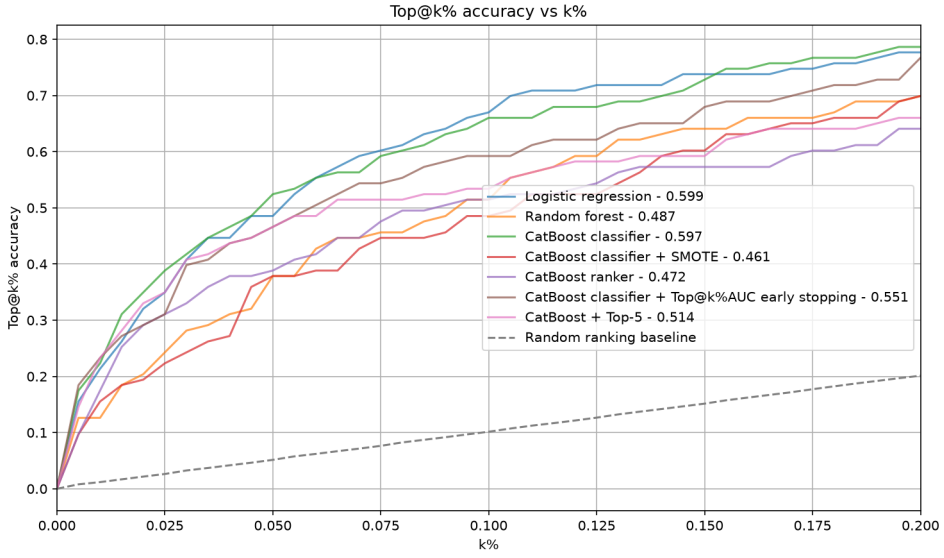


FIGURE 5. Top@k% curves and Top@k%AUC on comparison validation. Logistic regression has the largest point estimate (0.599) and is selected for the test portfolio; its difference from CatBoost is inconclusive.

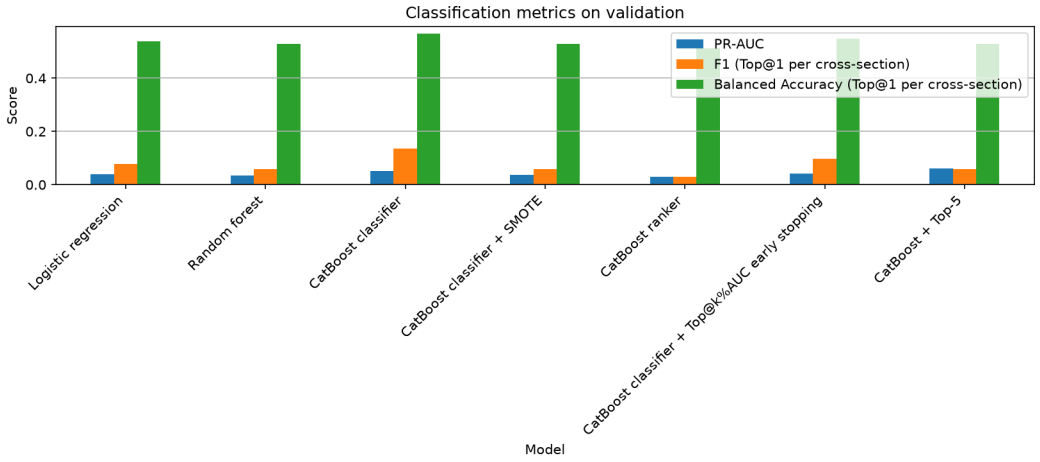


FIGURE 6. PR-AUC, F1, and balanced accuracy on the validation sample. F1 and balanced accuracy use the Top@1-per-cross-section decision rule.

TABLE VI. Top@k accuracy of models on test sample

Model	Top@1	Top@2	Top@5	Top@10	Top@20	Top@30
Logistic regression	0.058	0.115	0.404	**0.596**	**0.827**	**0.827**
Random forest	0.096	0.154	0.250	0.423	0.673	0.788
CatBoost classifier	0.192	**0.365**	**0.462**	0.558	0.673	0.769
CatBoost + SMOTE	0.096	0.192	0.327	0.385	0.596	0.654
CatBoost ranker	0.038	0.135	0.346	0.462	0.538	0.654
CatBoost + ES	**0.212**	0.308	0.365	0.558	0.750	0.788
CatBoost + Top-5	0.135	0.212	0.308	0.481	0.596	0.635

(0.827). These are point-estimate comparisons; significance is assessed separately in Section IV-I. Fig. 7 reports the corresponding curves.

All trained models outperform the random-ranking baseline, indicating that the feature set contains predictive information about future manipulation targets. Top@k values for the random-ranking model are computed as follows:

$$\text{Top@k}_{\text{random}} = \frac{1}{N}\sum_{i=1}^{N}\frac{k}{T_i} \tag{7}$$

where N is the number of test events and T_i is the i-th cross-section size. The observed hit rates exceed this random-ranking reference. The return ranker's test Top@k%AUC is 0.591, below the leading classifiers; possible mechanisms are discussed below.

C. TOP@K% ACCURACY

Table VII reports the corresponding percentage thresholds. CatBoost leads Top@1% (0.404); logistic regression leads Top@2% (0.500), Top@5% (0.769), Top@10% (0.827), Top@20% (0.904), and Top@50% (0.981). The aggregate ranking differences are assessed in Section IV-I.

D. TOP@K%AUC

In order to control for the problem of various cross-section sizes, we want to compute Top@k% instead of regular Top@k. Taking the normalized area under the Top@k% curve over $k\% \in (0, 20\%]$ allows us to compare models in the economically relevant low-portfolio-size region, consistent with Eq. 6. In Fig. 8 we compare the models based on this metric.

E. CLASSIFICATION METRICS

We complement ranking metrics with PR-AUC, F1 under the Top@1-per-event rule, and balanced accuracy (Fig. 9). CatBoost + ES has the largest test point estimates for all three: 0.1636, 0.2115, and 0.6044, respectively; CatBoost follows at 0.1009, 0.1923, and 0.5948. Under one true and one selected asset per event, precision, recall, and F1 all equal Top@1. None of their paired differences survives the 84-hypothesis Holm correction; PR-AUC and balanced accuracy are descriptive and were not hypothesis-tested.

F. ANALYSIS OF SMOTE PERFORMANCE

CatBoost + SMOTE attains test Top@1 of 0.096 and Top@k%AUC of 0.612, versus 0.769 for logistic regression. Its AUC deficit is significant against logistic regression and CatBoost on comparison validation, but not on test after the full Holm correction. The following properties provide plausible explanations for the point-estimate pattern rather than a causal attribution.

1) **Bounded and sign-meaningful features.** Several of our features—flow imbalance, slippage imbalance,

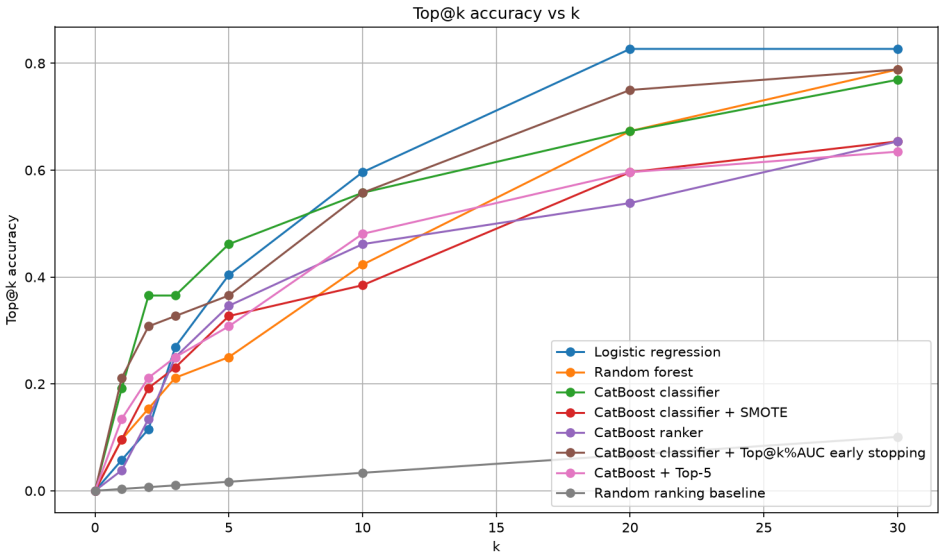


FIGURE 7. Test Top@k for all seven tuned models and the random-ranking baseline. The horizontal axis is portfolio size k; the vertical axis is the fraction of events whose manipulation target is among the k highest scores.

TABLE VII. Top@k% accuracy of models on test sample

Model	Top@1%	Top@2%	Top@5%	Top@10%	Top@20%	Top@50%
Logistic regression	0.327	**0.500**	**0.769**	**0.827**	**0.904**	**0.981**
Random forest	0.212	0.327	0.615	0.788	0.885	0.942
CatBoost classifier	**0.404**	0.481	0.635	0.769	0.885	0.962
CatBoost + SMOTE	0.250	0.365	0.500	0.654	0.808	0.962
CatBoost ranker	0.308	0.365	0.500	0.654	0.731	0.923
CatBoost + ES	0.346	0.442	0.635	0.788	0.885	0.962
CatBoost + Top-5	0.250	0.385	0.558	0.635	0.692	0.750

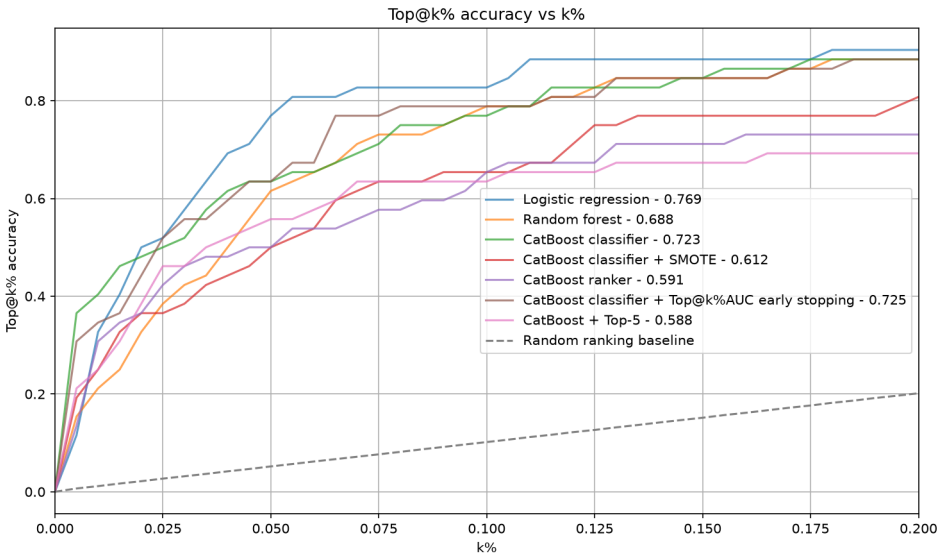


FIGURE 8. Top@k% accuracy on the test sample, normalizing the portfolio size by the cross-section size to control for its growth over time. The horizontal axis is the fraction k% of the cross-section retained; the vertical axis is the probability that the target is covered.

share of buy-initiated trades—are naturally bounded in $[-1, 1]$ (or $[0, 1]$) and are informative primarily near the positive extreme (e.g., heavy buy-side imbalance just before a pump). Interpolating between two pumped

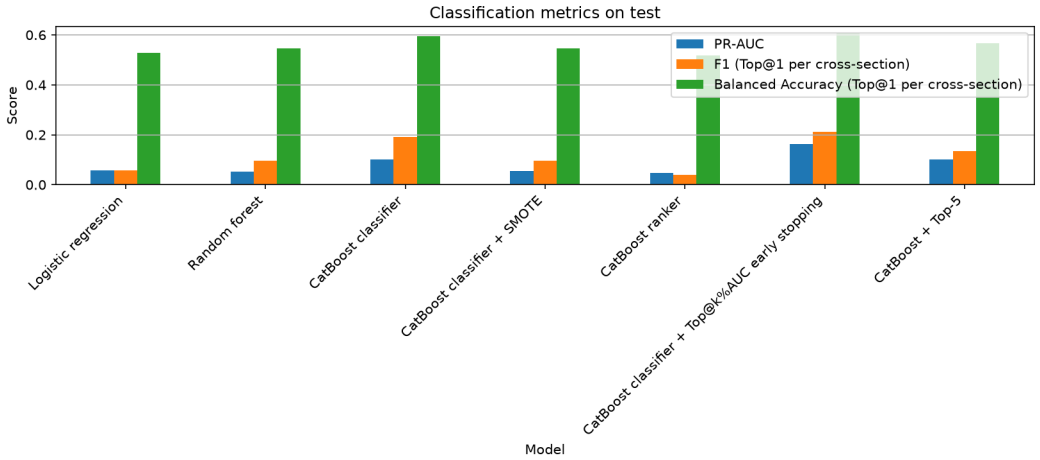


FIGURE 9. Test classification metrics for all seven tuned models: PR-AUC, F1 under Top@1 per event, and balanced accuracy. The test imbalance is approximately 1:297. These bars show point estimates.

samples whose imbalance is $+0.85$ and another whose imbalance is -0.20 yields a synthetic "pump" with imbalance $+0.32$, which lies inside the bulk of the negative-class distribution. SMOTE therefore dilutes rather than reinforces the class-discriminative region of the feature space.

2) **Cross-sectional reference.** The unbounded return, standardized-volume, and tail-shape features are standardized within the event cross-section (Eq. 5); for those columns, a value of $+2$ means "two cross-sectional standard deviations above the same-event average." Bounded ratios and count features retain their direct scales but are still interpreted jointly with those relative columns. SMOTE's nearest-neighbor interpolation mixes complete feature vectors from *different* events, producing synthetic samples whose relative and absolute components have no single corresponding cross-section in which they are coherent. The synthetic samples can be plausible marginally but out-of-distribution jointly with respect to the panel structure.
3) **High-dimensional sparsity.** With 81 features and only 310 positive training samples, the nearest-neighbor graph that SMOTE relies on is sparse and noisy: each minority point has few true neighbors in its class, and interpolation typically bridges across different subpopulations (e.g., small-cap vs. medium-cap pumps). This is the regime in which Blagus and Lusa [21] showed that SMOTE is at best neutral and frequently *degrades* classification performance.

Class-weight reweighting does not synthesize or perturb feature values; it changes gradient contributions on the observed samples. This difference is consistent with the point-estimate pattern, but the experiment does not isolate which of the proposed mechanisms explains SMOTE's performance.

G. ANALYSIS OF RANKING MODEL PERFORMANCE

The return ranker attains test Top@k%AUC of 0.591 and Top@1 of 0.038, compared with 0.725 and 0.212 for CatBoost + ES and 0.769 and 0.058 for logistic regression. The Top-5 hinge model attains AUC 0.588 and Top@5 0.308. Learning-to-rank need not dominate merely because evaluation uses ranks; the following properties may contribute to these differences.

Target-signal density. In the cross-sectional LTR setup of [14], every asset in the cross-section carries an informative continuous return target, so the listwise loss receives dense, ordinally-meaningful supervision. In our setup, exactly one asset per cross-section is a true manipulation target. All other assets contribute noise to the ordering: their 5-minute post-announcement returns are driven by background market movements, independent news, or incidental volatility, none of which the model is asked to predict. The ranker therefore spends most of its gradient on nuisance ordering among non-pumped assets, while the classifier only has to separate the single pump from the rest.

Loss-function alignment with the Top@k evaluation objective. Pairwise and listwise ranking losses (YetiRank, PairLogit, etc.) integrate pairwise or position-weighted comparisons across the entire list. When the positive cardinality is 1, most of those pairs are negative-vs.-negative and therefore non-informative for Top@k. The classification log-loss, conversely, concentrates gradient mass on the decision boundary between the single positive and the $\sim$297 negatives, which is exactly the boundary that Top@k accuracy measures. Class-weight reweighting further tilts the log-loss toward the positive class, yielding a gradient that is sample-efficient in the 1-vs.-many regime.

Target-construction noise. The continuous ranking target (realized arithmetic return over the first five minutes after announcement) is additionally noisy because (i) coincident market-wide moves can push a non-pumped asset's five-minute return above the pump target's, and (ii) micro-cap or-

der books produce large relative returns on very small trades, which the ranker has no way to discount. In the classification formulation the binary label is immune to this type of noise.

These mechanisms offer possible explanations for the return ranker's weaker aggregate performance in this sample; they are not independently tested causal conclusions.

The test AUC point estimates are logistic regression 0.769, CatBoost + ES 0.725, CatBoost 0.723, random forest 0.688, SMOTE 0.612, the return ranker 0.591, and Top-5 0.588. This ordering does not change the comparison-selected portfolio model and should be read together with the uncertainty estimates in Section IV-I.

H. INTERPRETATION OF THE MODELS

For interpretation we retain CatBoost + ES as a nonlinear diagnostic model, although logistic regression is selected for portfolios. CatBoost PredictionValuesChange importance (Fig. 10) ranks *num_prev_pump* first, followed by *num_trades@7D*, *num_trades@14D*, *asset_return@4H*, and *asset_return_zscore@4H*. These describe prior pump history, trading activity, and pre-announcement returns; importance alone does not establish causality.

To verify that this is not an artifact of a single fit, we recomputed the feature importances inside each of the 25 random 70% training-subset retraining runs of Section IV-J, which yields a training-subset distribution of importance for every feature. Table VIII reports the per-feature percentiles of the 15 most important features, sorted by median. Across subsets, *num_prev_pump* has the highest median importance, followed by *num_trades@7D*, *num_trades@2D*, *num_trades@14D*, and *asset_return@4H*; thus the broad prior-history and activity pattern is stable even though the precise ordering changes across fits.

I. STATISTICAL SIGNIFICANCE

We draw 10,000 bootstrap samples of whole pump events with replacement, separately for the 103 comparison and 52 test events, using seed 42. Every replicate contains the original number of events and uses identical draws across fixed models. Repeated pumps contribute their complete cross-section repeatedly. Predictions are computed once and reused; individual asset rows are never resampled independently. Table IX gives pointwise 95% percentile intervals; Fig. 11 displays the test intervals and paired decisions.

For every model pair, let $\widehat{\Delta}$ be its observed metric difference and Δ_b^* its difference on bootstrap draw b. We use a two-sided centered-null test,

$$p = \frac{1 + \sum_{b=1}^{B} \mathbf{1}\{|\Delta_b^* - \widehat{\Delta}| \geq |\widehat{\Delta}|\}}{B+1}, \qquad B = 10{,}000, \quad (8)$$

with an add-one Monte Carlo correction. Holm adjustment controls the comparison family of 21 model pairs for AUC, and the test family of 21 pairs × four metrics (AUC, F1, precision, recall; 84 hypotheses). The three classification metrics coincide under Top@1, but all remain in the stated correction family. Decisions use adjusted $p < 0.05$, not overlap of marginal intervals.

On comparison validation, logistic regression exceeds random forest ($p_{Holm} = 0.0240$), SMOTE (0.0187), and the return ranker (0.0072); CatBoost also exceeds those three (0.0021, 0.0038, and 0.0021, respectively). The other 15 pairs are inconclusive, including logistic regression versus CatBoost. Thus the selected model is the point-estimate leader, not a unique statistical winner.

On test, only three AUC comparisons remain significant: logistic regression minus the return ranker, $\Delta = 0.1781$, pointwise 95% interval $[0.0882, 0.2743]$, $p_{Holm} = 0.0252$; logistic regression minus Top-5, $\Delta = 0.1805$, $[0.0817, 0.2851]$, $p_{Holm} = 0.0415$; and CatBoost + ES minus the return ranker, $\Delta = 0.1346$, $[0.0649, 0.2118]$, $p_{Holm} = 0.0492$. Logistic regression versus CatBoost has $\Delta = 0.0459$, $[-0.0313, 0.1281]$, $p_{Holm} = 1.000$. The remaining AUC comparisons and all F1/precision/recall comparisons are inconclusive. Non-rejection is not equivalence; neither model ordering nor profitable PnL is established universally by these tests.

Inference assumes independent, exchangeable pump events and conditions on the fitted models. It excludes retraining and tuning uncertainty; serial dependence or shared organizers could reduce the effective sample size. Earlier reuse of these periods further limits prospective interpretation.

J. ROBUSTNESS ANALYSIS

1) Training data perturbation

As a separate diagnostic, we retrain CatBoost + ES 25 times on random 70% subsets of the training events (217 of 310; subset seeds 42–66), with the tuning, comparison, and test partitions fixed. This measures sensitivity to training-event composition conditional on the preprocessing and hyperparameters; it is not a robustness estimate for the selected logistic portfolio model. Table X summarizes test metrics.

CatBoost + ES Top@k%AUC has standard deviation 0.021 across these fits, with the central 80% in $[0.713, 0.766]$ (Fig. 12). These are descriptive retraining variations, distinct from event-bootstrap confidence intervals for the fixed full-training model.

2) Temporal subperiod analysis

The test set spans August 2021 to August 2022. We use June 1, 2022 as a fixed descriptive subperiod boundary and evaluate the comparison-selected logistic model.

The early subperiod, before June 1, 2022, contains 46 events and has Top@k%AUC 0.746. The late subperiod contains only six events and is excluded by the `min_pumps=10` guard. This sample therefore does not establish performance stability across bull and bear regimes.

V. PORTFOLIO STRATEGY

This section evaluates a portfolio strategy inspired by [16], with modified construction and execution assumptions tailored to the proposed scoring framework. Authors of [16]

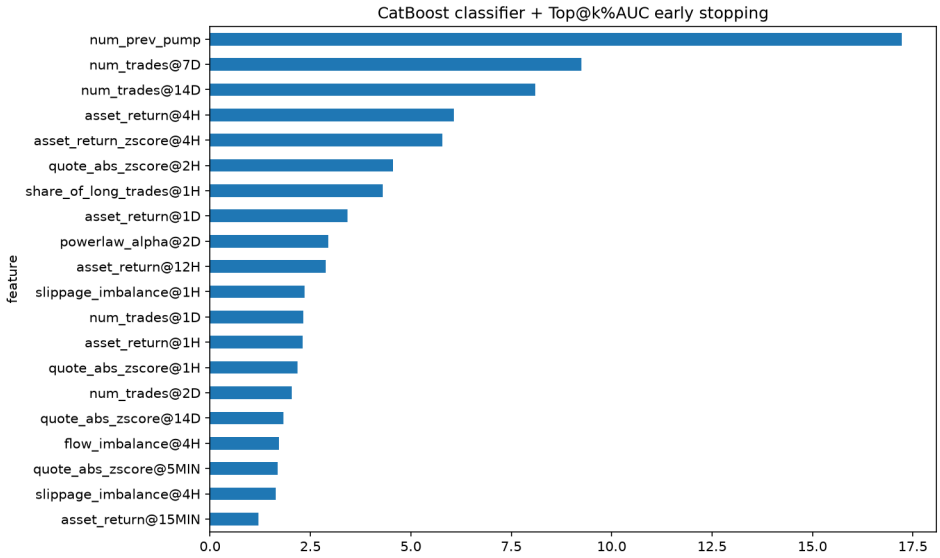


FIGURE 10. CatBoost + ES PredictionValuesChange importance for the full-training fit. The leading features are prior pump history, 7-day and 14-day trade counts, four-hour return, and four-hour standardized return. This diagnostic does not select the portfolio model.

TABLE VIII. Training-subset distribution of CatBoost + ES feature importances (PredictionValuesChange) across the 25 random 70% training-subset retraining runs of Section IV-J. The 15 features with the highest median importance are shown; columns report per-feature percentiles and rows are sorted by median.

Feature	10%	25%	Median	75%	90%
num_prev_pump	8.17	9.50	12.35	13.76	15.97
num_trades@7D	3.52	4.20	5.12	6.64	7.96
num_trades@2D	3.13	3.61	4.55	5.75	6.61
num_trades@14D	2.68	3.36	4.35	5.01	6.92
asset_return@4H	2.25	2.51	3.67	4.50	6.09
asset_return@1D	1.64	1.94	3.48	3.77	4.54
asset_return_zscore@12H	1.69	2.14	2.53	3.32	4.11
asset_return_zscore@4H	1.27	2.06	2.49	2.90	3.60
share_of_long_trades@1H	1.07	1.74	2.41	3.62	4.13
slippage_imbalance@12H	1.47	2.04	2.36	3.14	4.01
quote_abs_zscore@1H	0.90	1.65	2.12	2.43	2.88
num_trades@1D	1.03	1.81	2.11	3.57	4.43
asset_return_zscore@1D	1.00	1.26	1.96	2.30	2.97
slippage_imbalance@2D	0.91	1.46	1.84	2.49	2.97
quote_abs_zscore@5MIN	0.89	1.15	1.66	2.25	2.57

TABLE IX. Top@k%AUC and 95% pointwise event-bootstrap intervals (10,000 draws per partition). Rows follow comparison AUC; intervals are not simultaneous and do not themselves test paired differences.

Model	Comparison AUC	95% CI	Test AUC	95% CI
Logistic regression	0.599	[0.524, 0.671]	0.769	[0.681, 0.846]
CatBoost classifier	0.597	[0.521, 0.670]	0.723	[0.626, 0.812]
CatBoost + ES	0.551	[0.471, 0.628]	0.725	[0.631, 0.813]
CatBoost + Top-5	0.514	[0.432, 0.595]	0.588	[0.474, 0.702]
Random forest	0.487	[0.411, 0.561]	0.688	[0.598, 0.772]
CatBoost ranker	0.472	[0.391, 0.553]	0.591	[0.479, 0.700]
CatBoost + SMOTE	0.461	[0.385, 0.537]	0.612	[0.507, 0.711]

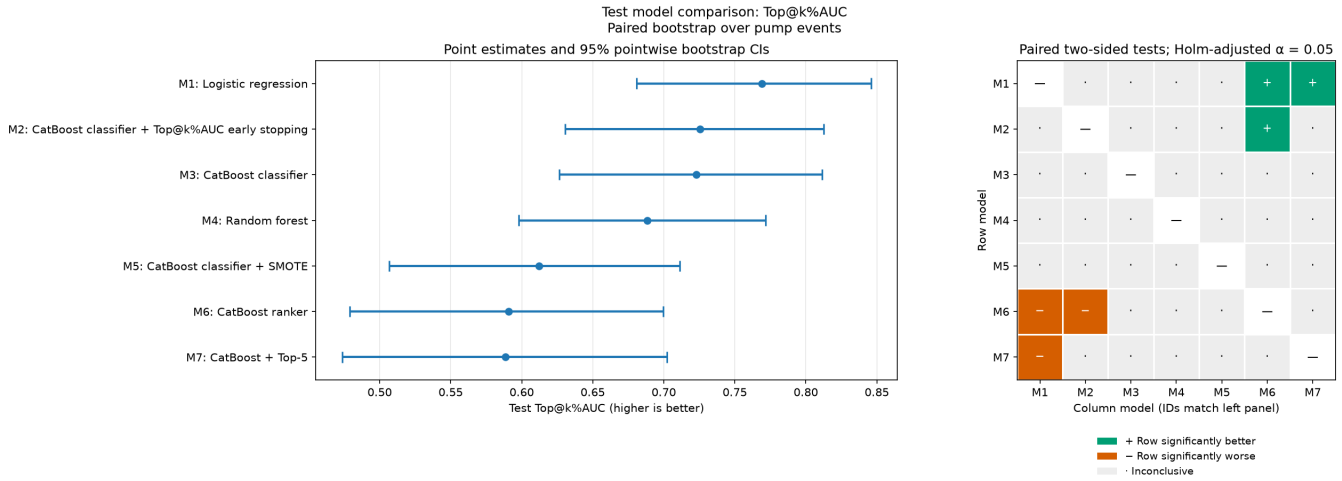


FIGURE 11. Test Top@k%AUC: point estimates and 95% pointwise intervals (left), and paired two-sided bootstrap decisions with Holm correction over all 84 test hypotheses (right). Models are ordered by test point estimate for display only. Green/orange indicates a significantly better/worse row model; a dot indicates an inconclusive comparison.

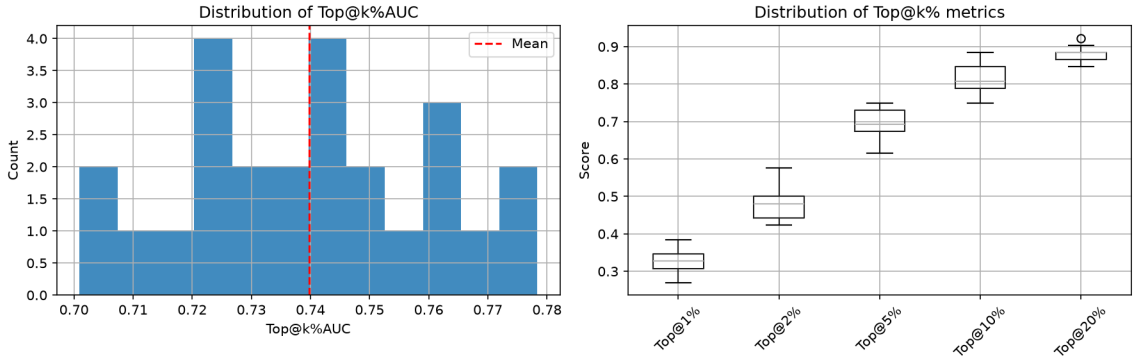


FIGURE 12. CatBoost + ES test metrics across 25 fits using different 70% training-event subsets. Tuning and test sets remain fixed. Left: Top@k%AUC histogram; right: Top@k% box plots. AUC standard deviation is 0.021 and its central 80% range is [0.713, 0.766].

TABLE X. CatBoost + ES test metrics across 25 random 70% training-subset fits. Top@k%AUC uses the normalized 0–20% range. The AUC standard deviation is 0.021.

Metric	Mean	Std	[10%, 90%]
Top@k%AUC	0.740	0.021	[0.713, 0.766]
Top@1%	0.328	0.029	[0.296, 0.365]
Top@2%	0.482	0.042	[0.442, 0.531]
Top@5%	0.695	0.041	[0.631, 0.742]
Top@10%	0.812	0.039	[0.750, 0.858]
Top@20%	0.883	0.017	[0.865, 0.904]

proposed using trained classification models to produce logits for each asset in the cross-section, then construct a portfolio of assets that have their logit of being pumped above some threshold. They also proposed using weights of the portfolio proportional to the logits produced by the model. They backtested their strategy using the following assumptions: assets that are not pumped do not change in price over the pump, authors buy at open price of the last hourly candle prior to the pump and are able to sell at half of pump's max return. They backtested their strategy using the test sample from October 29, 2018 to January 11, 2019 and were able to achieve a return of 60% over approximately two months. We extend this approach by constructing portfolios from the proposed model scores and by adopting more conservative execution assumptions.

We propose using models tuned for Top@k%AUC which will ensure that the models are optimized not only for Top@1 accuracy but for any arbitrary values of k. We relaxed some assumptions of [16] in the way we trade our portfolios, the timeline of how we trade each pump is described below:

1) Rank tickers by model logits (descending).
2) Select the Top@k set of tickers.
3) Submit equal-weight entries 15 minutes before the announcement and fill each at its first subsequent pre-announcement trade; omit a leg if no such trade occurs. The omitted leg's fixed $1/k$ weight remains in cash and is not redistributed.
4) Hold until the announcement; if not pumped, sell at the

announcement; if pumped, sell 1 minute after the pump.
5) Liquidate the complete acquired base-asset quantity, convert the BTC proceeds to USDT at the exit-time completed indicative rate, and subtract a 25 bp (0.25%) round-trip cost from entry capital.

We use logistic regression as the portfolio model selected on comparison validation. The execution-aware sensitivity analysis keeps $k = 5$ fixed; test results for $k \in \{1, 2, 5, 10, 20, 30\}$ are descriptive concentration checks and do not retune the strategy.

Table XI and Fig. 13 show a Sharpe ratio of 2.29 for the fixed $k = 5$ logistic portfolio. Among the descriptive sizes, $k = 20$ has the largest Sharpe (2.68), whereas $k = 1$ has the largest no-reinvestment cumulative return (1.303) and Sharpe 1.65. The six cumulative returns range from 0.155 to 1.303, compared with BTC/USDT's -49.7%. These are simulated point estimates, not significance tests of profitability.

For market context, the continuous BTC/USDT benchmark runs from August 24, 2021 14:45:03 UTC to August 14, 2022 16:01:05 UTC, matching the first decision and last strategy exit. It uses completed one-minute VWAPs and daily observations between those exact endpoints. BTC/USDT has cumulative return -0.497, annualized return -0.511, annualized volatility 0.679, and Sharpe -0.751. This is a calendar-aligned, not exposure-matched, benchmark: BTC remains invested between events.

A. EXECUTION-AWARE BACKTESTING WITH PRICE IMPACT

The results above use the first observed trade at or after each execution threshold but do not otherwise charge size-dependent slippage, which is unrealistic for the illiquid small-cap tokens typically targeted by P&D operations. To assess practical feasibility, we extend the backtesting framework with a fitted market-impact model that produces *size-dependent, dynamic* slippage in addition to the fixed round-trip fee. The model serves two purposes: first, it estimates a volume-weighted average price (VWAP) for both entry and exit; second, the shape of the impact curve is itself the *liquidity constraint* of the strategy—larger orders remain executable, but the fitted $\beta\sqrt{Q}$ curve makes each additional unit of notional arrive at progressively worse prices, so the economically relevant question shifts from "is the order executable?" to "at what order size does impact erode the signal?".

1) Impact model specification

We model price impact in basis points as a function of order notional Q (in USDT):

$$I(Q) = \beta\sqrt{Q}. \tag{9}$$

The square-root specification is a well-established empirical regularity in market microstructure. Kyle [25] introduced the foundational model of linear price impact from informed order flow; subsequent work demonstrated that the aggregate impact of executed orders follows a concave, approximately square-root, relationship with order size. Almgren and Chriss [19] formalized this within the optimal execution framework, while Almgren et al. [20] provided direct empirical estimates of the power-law exponent (≈ 0.6) using institutional equity data. Bouchaud et al. [22] surveyed the evidence for the square-root law across asset classes, and Tóth et al. [28] demonstrated its universality across stocks and time periods using a large meta-order dataset. In cryptocurrency markets specifically, Donier and Bonart [23] confirmed the square-root law in Bitcoin using over one million reconstructed meta-orders, showing that the relationship holds across four decades of order sizes.

We fit a single coefficient $\beta \geq 0$ via constrained OLS (no intercept) for each asset, pooling absolute impacts from both buy and sell candles. The entry regression uses the 14 days of 5-minute candles ending strictly at the actual simulated entry; it therefore cannot observe any later trade. For the manipulated asset, the execution backtest fits 5-second sell-dominated candles from T up to, but excluding, the realized exit timestamp, capped at $T + 10$ minutes. In each case, absolute net volume serves as Q and the absolute close-to-close price change (in bps) serves as the impact observation. Entry notionals are converted to USDT using the most recent completed BTC/USDT one-minute bar at entry; the exit model uses the completed rate available at the announcement. Neither conversion falls forward to a future or incomplete bar.

2) Impact measurement from candle data

Entry model (5-minute candles). We aggregate the 14-day trade history ending strictly at the actual simulated entry timestamp into 5-minute candles. For each candle we compute the net buy volume $Q_{\text{net}} = 2 \times V_{\text{taker buy}} - V_{\text{total}}$ (in quote currency), where $V_{\text{taker buy}}$ is the taker-buy quote volume and V_{total} is the total quote volume. The absolute net volume $|Q_{\text{net}}|$ serves as the order size, and the absolute close-to-close price change $|p_{\text{close}}/p_{\text{prev close}} - 1| \times 10^4$ bps serves as the impact observation. Observations from both buy- and sell-dominated candles are pooled into a single regression, which doubles the effective sample size and produces a more stable estimate of β. The 5-minute resolution matches the intended execution horizon: we assume the order is worked over a single candle interval, so the candle-level impact directly estimates the cost of execution.

Exit model for manipulated assets (5-second candles, sell-only). For the pumped asset, the backtest fits a separate model using only sell-dominated candles (negative Q_{net}) in $[T, \min(T + 10\text{min}, t_{\text{exit}}))$. The right-open exit bound prevents post-exit observations from entering the simulated execution cost. The full 10-minute window is used only for the illustrative regression figure below.

We model liquidity through a continuous price-impact penalty rather than a hard participation cap. This makes larger orders executable only at increasingly adverse VWAPs. A hard order-book depth or participation-rate cap would further

Portfolio / benchmark	Avg. event return	Cumulative return	Annualized portfolio return	Annualized portfolio volatility	Sharpe ratio
$k = 1$	0.0251	1.3031	1.3360	0.8079	1.6537
$k = 2$	0.0087	0.4498	0.4612	0.4230	1.0903
$k = 5$	0.0110	0.5698	0.5842	0.2548	2.2931
$k = 10$	0.0077	0.3995	0.4096	0.1610	2.5446
$k = 20$	0.0058	0.3034	0.3111	0.1161	2.6784
$k = 30$	0.0030	0.1555	0.1594	0.0804	1.9827
BTC/USDT buy-and-hold	—	-0.4966	-0.5105	0.6794	-0.7514

TABLE XI. Performance by portfolio size and the time-aligned BTC/USDT benchmark

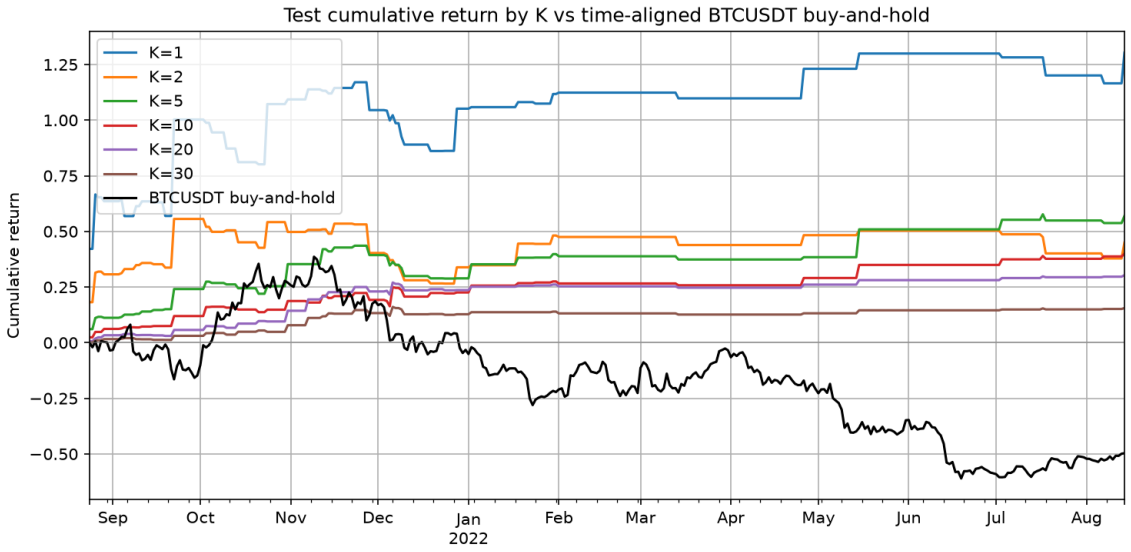


FIGURE 13. Test cumulative return paths for $k \in \{1, 2, 5, 10, 20, 30\}$ portfolios ranked by comparison-selected logistic regression, with a continuous BTC/USDT benchmark over the same calendar interval. Portfolio returns accumulate without reinvestment and include a 25 bp round-trip cost. The benchmark is black.

reduce realized returns and is therefore a conservative extension for future work.

3) VWAP entry and exit price estimation

The execution simulation proceeds as follows for each selected asset:

Step 1: Determine raw prices. Once the model has scored the cross-section at $t_{\text{pump}} - \Delta t_{\text{buy}}$, the entry price p^{entry} is the first trade price at or after that decision timestamp and strictly before t_{pump}. If no such trade occurs, the leg is not executable and is omitted. The exit price p^{exit} is the first trade price at or after t_{pump} (or $t_{\text{pump}} + \Delta t_{\text{sell}}$ for the manipulated asset). Thus no simulated fill predates the signal that generated it.

Step 2: Resolve intended notional. The target order size Q_{intended} is specified in USDT and converted to quote currency (BTC) using the indicative BTC/USDT rate at entry time. This normalization ensures that intended position sizes are comparable across BTC price regimes.

Step 3: Track inventory and exit notional. Let $R^{\text{entry}}_{\text{BTC/USDT}}$ denote the completed indicative conversion rate at entry. The entry quote notional and acquired base-asset quantity are

$$Q^{\text{entry}}_{\text{BTC}} = Q_{\text{intended}} / R^{\text{entry}}_{\text{BTC/USDT}}, \qquad B = Q^{\text{entry}}_{\text{BTC}} / p^{\text{entry}}_{\text{vwap}}. \tag{10}$$

The exit liquidates all B units. Its impact is evaluated at the contemporaneous raw quote notional $Q^{\text{exit}}_{\text{BTC}} = Bp^{\text{exit}}$, rather than reusing the entry BTC notional. Internally the impact functions use USDT-normalized sizes $\widetilde{Q}^{\text{entry}} = Q^{\text{entry}}_{\text{BTC}} R^{\text{entry}}_{\text{BTC/USDT}}$ and $\widetilde{Q}^{\text{exit}} = Q^{\text{exit}}_{\text{BTC}} R^{T}_{\text{BTC/USDT}}$, where R^T is the completed announcement-time rate used to fit the manipulated exit model. This prevents exit slippage from being understated when the target price rises. We assume that deeper layers of the order book remain available beyond the volume observed in historical candles. Larger trades therefore remain feasible, but they incur larger slippage through the impact model.

Step 4: Compute VWAP execution prices. The fitted coefficient estimates the price displacement associated with a given net order flow. An order that walks through the book continuously fills at improving prices along the way.

Integrating the square-root impact profile over the execution path yields the average fill cost:

$$I_{\text{vwap}}(Q) = \frac{1}{Q}\int_0^Q \beta\sqrt{v}\,dv = \tfrac{2}{3}\,\beta\sqrt{Q}. \tag{11}$$

The VWAP impact is two-thirds of the terminal impact because early fills occur at better prices than the final fill level. The VWAP execution prices are then:

$$p^{\text{entry}}_{\text{vwap}} = p^{\text{entry}}\left(1 + I_{\text{vwap}}(\widetilde{Q}^{\text{entry}})/10^4\right), \tag{12}$$

$$p^{\text{exit}}_{\text{vwap}} = p^{\text{exit}}\left(1 - I_{\text{vwap}}(\widetilde{Q}^{\text{exit}})/10^4\right). \tag{13}$$

Buying pushes the fill price above the initial quote; selling pushes it below. Let $R^{\text{exit}}_{\text{BTC/USDT}}$ be the completed exit-time conversion rate. The fully USDT-denominated transaction return is

$$r_{\text{USDT}} = \frac{B p^{\text{exit}}_{\text{vwap}} R^{\text{exit}}_{\text{BTC/USDT}}}{Q_{\text{intended}}} - 1 - 0.0025, \tag{14}$$

where the 25 bps term is charged against entry capital and covers the round-trip exchange fee. This accounting includes the BTC/USDT move over the holding interval and never mixes a BTC-denominated asset return with an exit-time USDT conversion.

Manipulated-asset exit. The sell leg occurs during the manipulation window where buying and selling pressure are not balanced. Its impact model uses 5-second sell-dominated candles available strictly before the exit, never observations after the simulated transaction. A manipulation-window fit is used only when at least 20 sell-dominated candle samples are available; otherwise the framework logs the sample count and falls back to the pre-pump entry model.

4) Regression fit quality

Fig. 14 shows an example of the fitted entry impact regression for one pump asset. The absolute-impact approach pools buy and sell 5-minute candles into a single regression ($I = \beta\sqrt{Q}$, no intercept), yielding $R^2 = 0.310$ on the buy side, $R^2 = 0.355$ on the sell side, and $\beta = 0.7224$ in the example shown.

Fig. 15 is a descriptive fit over the full capped 10-minute manipulation window for one asset. This longer diagnostic window is used only to visualize the sell-side relation; the execution backtest instead truncates each fit at that transaction's realized exit. The sell-only 5-second candle approach yields $R^2 = 0.308$ with $\beta = 0.8997$ ($n = 71$ candles) in the example shown. The higher β reflects the increased cost of selling during a pump, consistent with the asymmetric liquidity conditions.

5) Unmodeled execution frictions

The fitted impact model captures average slippage against historical candle-level liquidity but intentionally omits several real-world frictions in order to keep the estimate interpretable. Specifically, we do not model (i) order-book depth exhaustion at the deepest levels (which would manifest as partial fills rather than a smooth $\beta\sqrt{Q}$ curve), (ii) queue priority and time-in-force dynamics on the exchange matching engine, (iii) exchange-side API latency between signal generation and order arrival, and (iv) the risk that other market participants detect the same pump signature and front-run the entry. Each of these frictions works in the same direction and would reduce realized PnL further, so the sensitivity numbers reported below should be read as an upper bound on achievable performance.

6) PnL sensitivity to order size

We sweep intended per-trade size from 100 to 10,000 USDT for the fixed $k = 5$ logistic portfolio, with a decision 15 minutes before announcement and pumped-asset exit one minute after. Across 52 test events, no-reinvestment cumulative ROE falls from 0.486 at 100 USDT to -0.261 at 10,000 USDT. It is 0.195 at 2,000 USDT and -0.020 at 5,000 USDT, crossing zero between those sizes (Fig. 16). These conditional simulations illustrate how the fitted impact penalty erodes returns; they do not establish live profitability or a statistically significant capacity threshold.

VI. LIMITATIONS AND FUTURE WORK

Exchange generalizability. Our analysis is restricted to Binance BTC pairs. P&D dynamics may differ on smaller exchanges with lower liquidity, different fee structures, or weaker surveillance. Validating the framework on additional exchanges is a natural extension, though it requires comparable data availability.

Dataset size. With 517 labeled events (52 in the test set), statistical power is limited. The bootstrap confidence intervals (Section IV-I) quantify this uncertainty, but larger labeled datasets would enable more precise performance estimation.

Execution assumptions. Although we incorporate a fitted price-impact model estimated from pre-pump 5-minute candles and post-pump 5-second sell candles (Section V-A), the backtesting framework does not model all real-world frictions, including exchange API latency, order-book dynamics during the pump itself, or the risk of front-running by other participants who detect the same signal.

Evolving manipulation tactics. Manipulators may adapt their strategies over time. The late subperiod is too small to establish cross-regime stability; evaluation on subsequently collected events is needed before deployment.

Alternative imbalance-handling approaches. While we compared class weighting and SMOTE, other techniques such as focal loss [26], cost-sensitive learning [24], and deep anomaly detection [27] remain unexplored in this setting and could potentially improve performance.

Feature audit. A historical lookahead in z-score normalizers was corrected by anchoring full hourly bars and all predictor windows to $T - 15$ minutes, followed by feature regeneration and separate tuning. The perturbation, provenance, and fitting-isolation checks described above found no predictor leakage in the tested paths. They cannot undo prior researcher inspection of these periods or independently

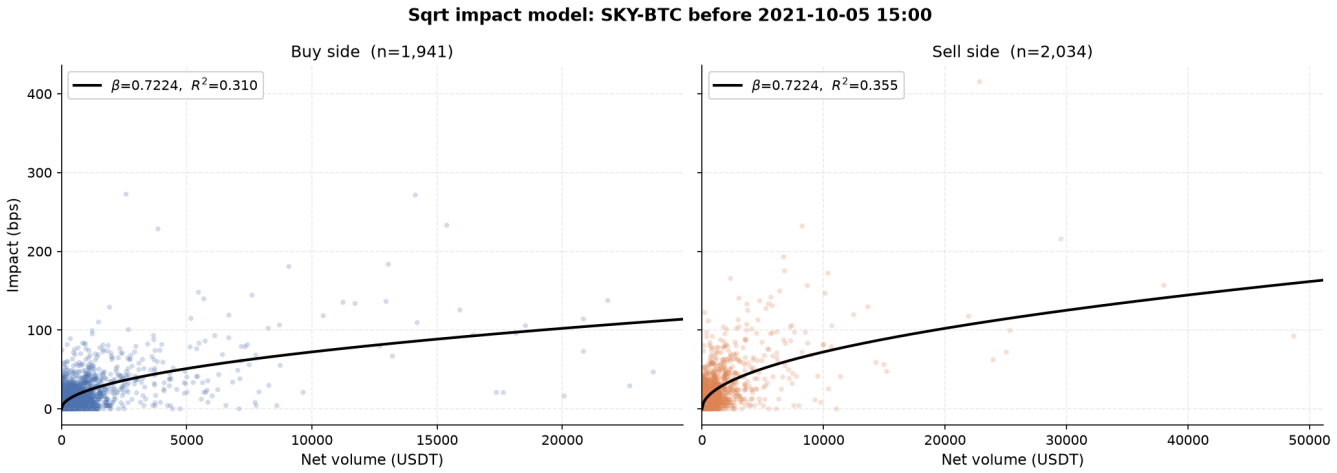


FIGURE 14. Fitted entry price-impact regression for the SKY-BTC pair, using the 14 trading days of 5-minute candles ending 15 minutes before the pump announcement. Left panel: candles with net buying pressure (positive Q_{net}); right panel: candles with net selling pressure (negative Q_{net}). Each scatter point is one 5-minute candle; horizontal axis $|Q_{\text{net}}|$ is absolute net volume in USDT and vertical axis $|\acute{p}|$ is the absolute close-to-close price change in basis points. Solid line: fitted $|\acute{p}| = \beta\sqrt{|Q_{\text{net}}|}$ (no intercept, $\beta = 0.72$), estimated on both panels jointly. Takeaway: the single-parameter square-root fit captures the concave impact–size relation on both sides of the book, consistent with the square-root impact law documented across asset classes [23], [28]; this is the curve used to translate order size into VWAP slippage in the backtest.

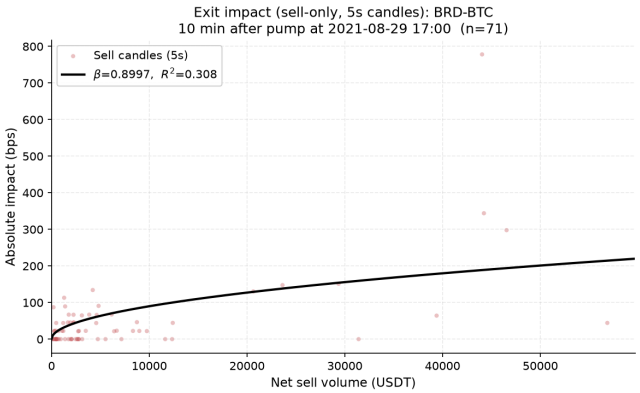


FIGURE 15. Descriptive exit-impact fit for BRD-BTC using sell-dominated 5-second candles in the first 10 minutes after announcement ($n = 71$, $\beta = 0.90$, $R^2 = 0.308$). This illustration uses a longer window than an executed exit: each backtested transaction uses only candles strictly preceding its own exit timestamp.

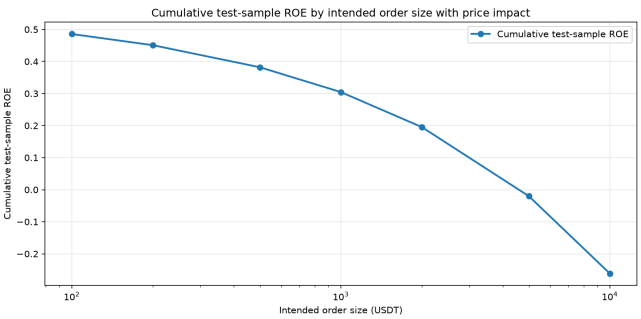


FIGURE 16. Impact-aware cumulative ROE over 52 test events for the fixed $k = 5$ logistic portfolio. Intended per-trade size ranges from 100 to 10,000 USDT; cumulative ROE falls from 0.486 to −0.261 and crosses zero between 2,000 and 5,000 USDT.

establish the correctness of every original event annotation; a fresh prospective sample remains necessary.

VII. CONCLUSION

This study compares seven tuned models for predicting pump-and-dump targets from strictly pre-decision microstructure features. In the current 517-event retrospective partition, comparison validation selects logistic regression, which attains test Top@k%AUC 0.769 with 95% interval $[0.681, 0.846]$. CatBoost + ES leads the descriptive test Top@1 and classification metrics. Holm-adjusted tests establish three test AUC differences, but no unique winner across alternatives and no significant classification-metric differences. The selected logistic portfolio's simulated returns decline with order size once fitted impact is charged.

Reproduction is supported by fixed split manifests, hashed features and fitted models, exported paired draws, and replay checks. Future-data perturbations and estimator-boundary tests explicitly check lookahead and regressor leakage. The findings support further evaluation for market surveillance, while their statistical uncertainty, prior period reuse, and execution assumptions require caution about prospective performance.

REFERENCES

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019.

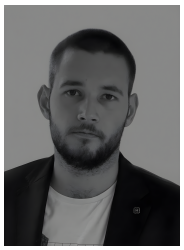
[2] J. Albers, M. Cucuringu, S. Howison, and A. Y. Shestopaloff, "Fragmentation, price formation and cross-impact in Bitcoin markets," *Appl. Math. Finance*, vol. 28, 2022.

[3] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, 2001.

[4] V. Chadalapaka, K. Chang, G. Mahajan, and A. Vasil, "Crypto pump and dump detection via deep learning techniques," *arXiv*, 2022.

[5] L. W. Cong, X. Li, K. Tang, and Y. Yang, "Crypto wash trading," *Manage. Sci.*, vol. 69, no. 11, 2023.

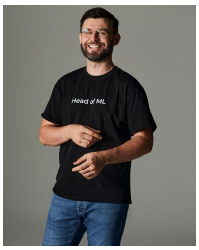
[6] D. Fantazzini and Y. Xiao, "Detecting pump-and-dumps with crypto-assets: Dealing with imbalanced datasets and insiders' anticipated purchases," *Econometrics*, vol. 11, no. 3, 2023.
[7] L. Grinsztajn, O. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems*, 2022.
[8] S. Hu, Z. Zhang, S. Lu, B. He, and Z. Li, "Sequence-based target coin prediction for cryptocurrency pump-and-dump," *Proc. ACM Manag. Data*, vol. 1, no. 1, 2023.
[9] J. Kamps and K. Bennett, "To the moon: Defining and detecting cryptocurrency pump-and-dumps," *Crime Sci.*, vol. 7, p. 18, 2018.
[10] M. La Morgia, A. Mei, F. Sassi, and J. Stefa, "Pump and dumps in the Bitcoin era: Real-time detection of cryptocurrency market manipulations," in *Proc. Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2020, pp. 1–9.
[11] M. La Morgia, A. Mei, F. Sassi, and J. Stefa, "The doge of wall street: Analysis and detection of pump-and-dump cryptocurrency manipulations," *ACM Trans. Internet Technol.*, vol. 23, no. 1, pp. 1–28, 2023.
[12] H. Nghiem, G. Muric, F. Morstatter, and E. Ferrara, "Detecting cryptocurrency pump-and-dump frauds using market and social signals," *Expert Syst. Appl.*, art. no. 115284, 2021.
[13] A. Ntakaris, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Mid-price prediction based on machine learning methods with technical and quantitative indicators," *PLOS ONE*, vol. 15, no. 6, 2020.
[14] D. Poh, B. Lim, S. Zohren, and S. Roberts, "Building cross-sectional systematic strategies by learning to rank," *J. Financial Data Sci.*, vol. 3, no. 2, pp. 70–86, 2021.
[15] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.
[16] J. Xu and B. Livshits, "The anatomy of a cryptocurrency pump-and-dump scheme," in *Proc. 28th USENIX Secur. Symp. (USENIX Security 19)*, 2019, pp. 1609–1625.
[17] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
[18] M. Mironov, D. Bogutsky, and P. Lukianchenko, "Pump-and-dump target prediction (code repository), GitHub, 2025. [Online]. Available: https://github.com/B0R0koko/pump-and-dump-prediction. Accessed on: Dec. 24, 2025.
[19] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *J. Risk*, vol. 3, no. 2, pp. 5–39, 2001.
[20] R. Almgren, C. Thum, E. Hauptmann, and H. Li, "Direct estimation of equity market impact," *Risk*, vol. 18, no. 7, pp. 58–62, 2005.
[21] R. Blagus and L. Lusa, "SMOTE for high-dimensional class-imbalanced data," *BMC Bioinform.*, vol. 14, p. 106, 2013.
[22] J.-P. Bouchaud, J. D. Farmer, and F. Lillo, "How markets slowly digest changes in supply and demand," in *Handbook of Financial Markets: Dynamics and Evolution*, T. Hens and K. R. Schenk-Hoppé, Eds. Elsevier, 2009, ch. 2.
[23] J. Donier and J. Bonart, "A million metaorder analysis of market impact on the Bitcoin," *Market Microstructure and Liquidity*, vol. 1, no. 2, art. 1550008, 2015.
[24] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2001, pp. 973–978.
[25] A. S. Kyle, "Continuous auctions and insider trading," *Econometrica*, vol. 53, no. 6, pp. 1315–1335, 1985.
[26] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980–2988.
[27] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
[28] B. Tóth, Y. Lemperiere, C. Deremble, J. de Lataillade, J. Kockelkoren, and J.-P. Bouchaud, "Anomalous price impact and the critical nature of liquidity in financial markets," *Phys. Rev. X*, vol. 1, art. 021006, 2011.



MIKHAIL MIRONOV received the B.S. degree in economics from Universitat Pompeu Fabra, Barcelona, Spain, and the National Research University Higher School of Economics (HSE), St. Petersburg, Russia, in 2023 (double degree), and the M.S. degree with honors in financial economics from International College of Economics and Finance, ICEF HSE, Moscow, Russia, in 2025. He is currently pursuing the Ph.D. degree in finance at HSE, Moscow, Russia. His major field of study is finance and financial economics.



DENIS BOGUTSKY received the M.S. degree in mathematics from Lomonosov Moscow State University in 2020. He is currently a Junior Researcher with the Laboratory of AI in Mathematical Finance at the National Research University Higher School of Economics. His research focuses on quantitative finance, market microstructure, credit risk modeling, and anomaly detection in financial systems.



PETR P. LUKIANCHENKO received the bachelor's and master's degrees in information science from HSE University, Moscow, Russia, in 2009 and 2011, respectively, and the Ph.D. degree in mathematics. He has been with HSE University, since 2009. He currently holds multiple positions, including a Senior Lecturer with the Faculty of Computer Science, Big Data and Information Retrieval School; a Lecturer with the Centre for Training Top AI Professionals; and the Head of the Laboratory of Artificial Intelligence in Mathematical Finance, AI and Digital Science Institute. At Trusted AI Research Center, he conducts research projects with a particular focus on image processing and time series data. He has co-authored research on neural-network-based approaches for predicting anomalies in interest rates under stochastic noise. He has presented his work on a multiagent simulator for financial crisis modeling at AI Journey 2025 Conference. His research interests include artificial intelligence applications in finance, including the detection of structural breaks in financial time series and the development of simulators for financial market crises.